

Note that BSBI is a multi-pass algorithm since it breaks up the document collection into multiple blocks, processes each block individually in a sorting phase, and then merges the sorted results of each block in the final step. The alternative to BSBI is the 'Single Pass In-Memory Indexing' (SPIMI). While BSBI processes each block and creates a sorted list of (term ID, doc ID) pairs, SPIMI does not have the pre-processing step of creating the sorted list of (term ID, doc ID) pairs.

Similar to BSBI, SPIMI also divides the collection into blocks. But the division into blocks is governed by the memory available. As long as there is enough free memory available, SPIMI continues to process one term at a time. On encountering a term, SPIMI checks to see if the term is already encountered as part of the current block's dictionary. If not, it creates a new term entry in the dictionary and creates a 'pos list' associated with that term. If the term is already present in the dictionary, it retrieves the 'pos list' associated with that term and adds this 'doc ID' to the 'pos list'. When there is not enough free memory, it stops the processing of terms, sorts the current terms dictionary, writes it to disk and starts the processing of the next block. After processing all the terms, there are a number of on-disk block dictionaries which need to be combined to create the final 'inverted index'.

Note that, till now, we have been discussing the creation of an inverted index for a document collection in a single computer system, and the only hardware constraint we have considered so far is the amount of available main memory in the computer system where the document collection is being processed. However, consider the case of constructing an inverted index for the document collection that comprises the 'World Wide Web'. This task cannot be done on a single computer system and needs to be carried out on a very large cluster of machines. Hence, a distributed 'inverted index' creation method is needed. This is typically done in search engines using the 'Map Reduce' paradigm, which I will cover in my next column.

In the earlier discussion on BSBI and SPIMI, we assumed that documents can be scanned to get a token stream, and each token can be considered as a term for the document collection. Similarly, a user query is also broken down into query terms, and we look for documents which contain the query terms using the inverted index. We have glossed over many of the issues that arise when creating the terms from documents or queries. Documents can typically contain many common words such as 'the', 'an', 'a', etc. Such words do not contribute to deciding which documents are more relevant to a user query. Hence, such common terms need to be eliminated from being added to the term dictionary. This is typically achieved by what is known as a 'stop word list'. Any word in the 'stop word list' is skipped from being processed when it is encountered in a document.

There are further complications in tokenisation. How do we handle hyphenated terms such as 'anti-discriminatory' or 'well-known', etc? Do we break them into individual pieces or treat them as a single term? How do we handle the case sensitivity of terms in document collection and in queries? Case sensitivity can be useful in distinguishing between two semantically different query terms such as 'Windows' and 'windows'. For instance, 'Windows' could potentially refer to the Windows operating

system, whereas 'windows' could refer to windows in a house. But maintaining case sensitivity in an inverted index makes the index bloat up and, hence, most search engines/information retrieval systems do not support case sensitivity.

So far we have been considering simple queries where the user types the exact query without any spelling mistakes. But, frequently, users do misspell the query terms. Misspelled queries need to be handled intelligently by information retrieval systems such as search engines. We have often seen search engines give us intelligent hints such as, "Did you mean 'carrot'?" when the user misspells 'carrot' as 'carot'. One way of handling misspelled queries is to compute the edit distance from the misspelled word to words closest to it in the dictionary, and then use that set of words as the search term. Misspelled queries become more difficult to detect and deal with for proper nouns, such as in cases like 'Tom Cruise' getting misspelled as 'Tom Cruise, etc. The major challenge is to identify the subset of words in the dictionary for which we want to compute the edit distance with the query term.

Another challenge for information retrieval systems is the issue of handling wild card queries. I leave it as a take-away question to the reader to come up with an algorithm for handling wild card queries.

My 'must-read book' for this month

This month's book suggestion comes from one of our readers, Nita Kumar. Her recommendation is the book, 'Beautiful Data: The Stories Behind Elegant Data Solutions' by Toby Segaran and Jeff Hammerbacher. It comprises a series of essays on elegant data solutions encompassing a wide spectrum of topics such as opinion polls data analysis, data analysis of Mars images and data storage on the cloud. By the way, Toby Segaran is the author of the famous book, 'Programming the Collective Intelligence', which discusses how to mine the data on the Web. Thank you, Nita for your recommendation.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group [Computer Science Interview Training India](http://www.linkedin.com/groups?home=HYPERLINK) at <http://www.linkedin.com/groups?home=HYPERLINK> <http://www.linkedin.com/groups?home=&gid=2339182> and <http://www.linkedin.com/groups?home=&gid=2339182>